2025-2026

VERİ MADENCİLİĞİ

Dr. Öğr. Üyesi Alper Talha Karadeniz

Hafta 2

01
Veri Tipleri ve Veri Yapıları

02
Veri Kalitesi Problemleri

03
Eksik Veriler, Gürültü ve Aykırı Değerler

04
Uygulama

05
Veri Temizleme Teknikleri

06
Normalizasyon ve Standardizasyon

07
Özellik Seçimi ve Dönüştürme

08
Uygulama

Veri Nedir?

Veri madenciliğinde kullanılan veri kümesi, genellikle veri nesnelerinden (object/instance) ve özelliklerden (feature/attribute) oluşan bir koleksiyondur.

Veri nesnesi: Gözlem birimidir. Her bir nesne, incelenen bir varlığı temsil eder (örneğin bir öğrenci, müşteri, ürün).

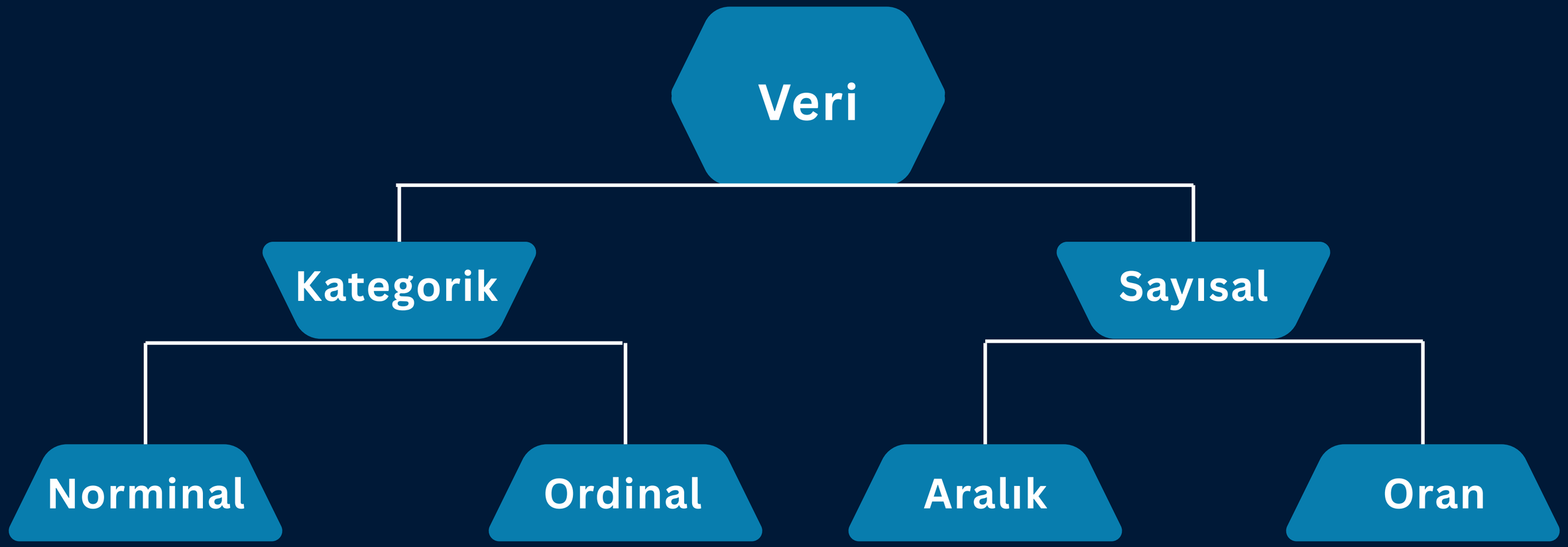
Özellik (feature): Veri nesnesini tanımlayan niteliklerdir. Her nesne, bu özelliklere göre ayırt edilir (örneğin yaş, cinsiyet, fiyat, kategori).

Bu yapı sayesinde algoritmalar, nesneler arasındaki desenleri ve ilişkileri analiz edebilir.

Özellikler

Nesneler

	A	B	C
1	**Product**	**Quantity**	**Total Sales**
2	Hoodie	30	1770
3	T-shirt	50	1450
4	T-shirt	45	1305
5	Sweater	100	8900
6	Hoodie	120	7080
7	Hoodie	155	9145
8	T-shirt	80	2320
9	Sweater	90	8010
10	T-shirt	60	2940
11	Sweater	80	7120
12	Sweater	120	10680


Veri
Kategorik
Sayısal
Norminal
Ordinal
Aralık
Oran

Veri Türleri - Kategorik

Nominal Veri Türü

Nominal veriler, yalnızca kategorilere veya gruplara ayrılmış, sayısal anlam taşımayan, yalnızca isimlendirme veya etiketleme için kullan verilerdir.
Bu tür verilerde sınıflar arasında herhangi bir sıralama veya derecelendirme yoktur.
Bu tür veriler, sayılarla ifade edilse bile sayılar yalnızca birer etiket görevi görür; matematiksel anlam taşımaz.

Örnekler:
Cinsiyet: Erkek, Kadın
Kan grubu: A, B, AB, 0
Renkler: Kırmızı, Mavi, Yeşil
Ülke isimleri: Türkiye, Fransa, Japonya

Ordinal Veri Türü

Ordinal veriler, belirli bir sıraya göre düzenlenmiş kategorilerdir.
Bu veri türünde sıralama vardır ancak kategoriler arasındaki farkların ne kadar olduğu bilinmez.
Yani bir kategori diğerinden “daha fazla” veya “daha az” olabilir, ama ne kadar daha fazla/az olduğu belli değildir.

Örnekler:
Memnuniyet düzeyi: Çok memnun, Memnun, Kararsız, Memnun değil, Hiç memnun değil
Eğitim seviyesi: İlkokul < Ortaokul < Lise < Üniversite
Yarış sıralaması: 1.'lik, 2.'lik, 3.'lük
Aciliyet düzeyi: Düşük, Orta, Yüksek

Veri Türleri - Sayısal

Interval (Aralık) Veri Türü

Interval veri türü, veriler arasında eşit aralıklar bulunan, ancak mutlak sıfır noktası olmayan veri türüdür.
Yani bu verilerle toplama ve çıkarma yapılabilir, ancak oran hesaplaması (çarpma ve bölme) yapılamaz çünkü “sıfır” eksiklik anlamına gelmez.
Sıfır noktası keyfidir.

Örnekler:
Sıcaklık (Celsius veya Fahrenheit): 30°C, 40°C. 0°C mutlak sıfır değildir. 40°C, 20°C’nin iki katı sıcaklık anlamına gelmez.
Tarih: 1990, 2000 yılları arasında 10 yıl vardır ama “2000 yılı, 1000 yılın iki katı değildir”.

Ratio (Oranlı) Veri Türü

Ratio veri türü, sayısal değerlerin eşit aralıklı olduğu ve aynı zamanda mutlak bir sıfır noktası içeren veri türüdür. Bu sayede hem toplama-çıkarma hem de çarpma-bölme işlemleri yapılabilir. En güçlü ve en anlamlı ölçüm düzeyidir. Tüm matematiksel işlemler (toplama, çıkarma, çarpma, bölme) geçerlidir.
Örnekler:
Uzunluk: 0 cm gerçekten hiç uzunluk yok demektir. 10 cm, 5 cm’nin iki katıdır.
Ağırlık: 0 kg = yok. 6 kg, 3 kg’nin iki katıdır.
Yaş: 0 yaş = doğmamış. 20 yaş, 10 yaşın iki katıdır.
Sıcaklık (Kelvin): Mutlak sıfır noktası var.

Kesikli ve Sürekli Değişkenler

KESİKLİ

- Kesikli değişkenler, sayma yoluyla elde edilen ve belirli, ayrık değerler alan değişkenlerdir.
  Genellikle tam sayı değerler alırlar ve iki değer arasında başka ara değer bulunmaz.
- Bu tür değişkenlerde veri noktaları birbirinden nettir ve ayrı ayrı gözlemlenir.
- Örnekler:
  - Öğrenci sayısı
  - Araba sayısı
  - Telefon çağrısı sayısı
  - Bir sınıftaki kişi sayısı

SÜREKLİ

- Sürekli değişkenler, belirli bir aralıkta sonsuz sayıda değer alabilen değişkenlerdir.
  Genellikle ölçüm yoluyla elde edilirler ve küsuratlı (ondalıklı) değerler alabilirler.
- İki değer arasında teorik olarak sonsuz sayıda ara değer bulunabilir.
  Bu nedenle değerler kesintisiz bir yapıdadır.
- Örnekler:
  - Boy uzunluğu
  - Ağırlık
  - Sıcaklık
  - Zaman
  - Mesafe

Veri Analizinde Kullanılan Temel Ölçüler

Merkezî Eğilim Ölçüleri

Merkezî eğilim ölçüleri, bir veri setindeki değerlerin topluca hangi noktada yoğunlaştığını, yani ortalama eğilimin hangi değere yakın olduğunu gösterir.

Aritmetik Ortalama: Tüm verilerin toplanıp, veri sayısına bölünmesiyle elde edilir. Verilerin genel eğilimini gösterir.

Medyan (Ortanca): Sıralı bir veri setinde ortada kalan değerdir. Aykırı değerlerden etkilenmez.

Mod (Tepe Değeri): En sık görülen değerdir. Özellikle nominal verilerde kullanılır.

Veri Analizinde Kullanılan Temel Ölçüler

Dağılım Ölçüleri

Dağılım ölçüleri, verilerin ne kadar yayıldığını ya da birbirine ne kadar uzak olduğunu gösterir. Merkezî eğilim ölçüleri tek başına yeterli olmadığında, veri dağılımını anlamak için kullanılır.

Varyans: Verilerin ortalamaya göre ne kadar sapma gösterdiğini gösteren ölçüdür.

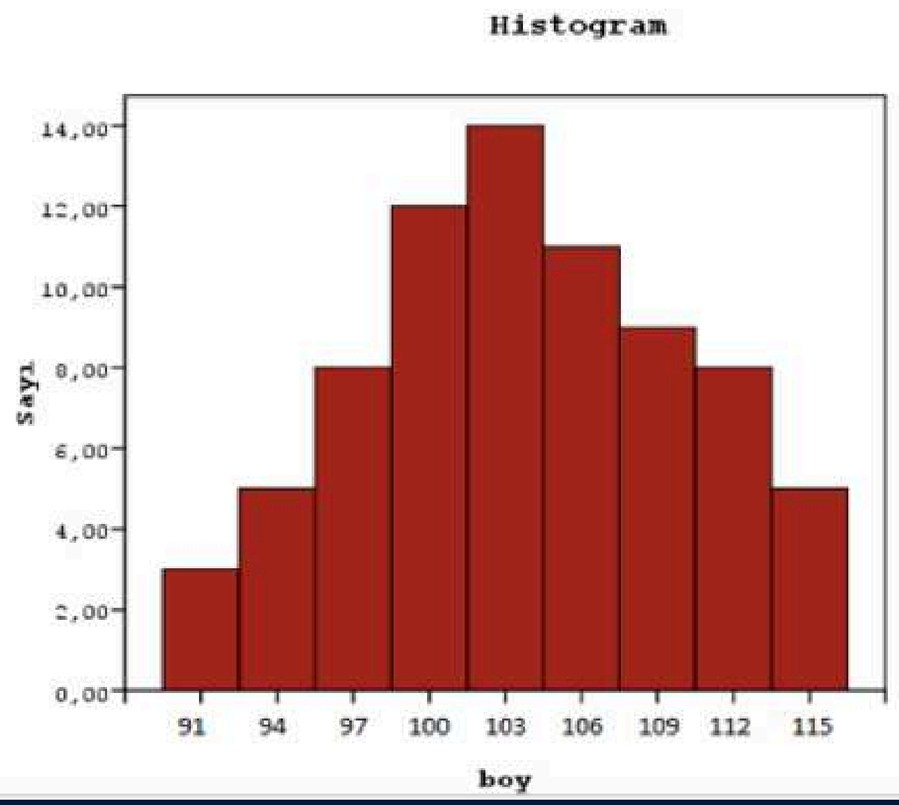
Standart Sapma: Varyansın kareköküdür. Verilerin ortalamaya olan ortalama uzaklığını verir.

Değişim Katsayısı: Standart sapmanın ortalamaya oranıdır, farklı ölçeklerdeki veri setlerini karşılaştırmak için kullanılır.

Çeyrekler Açıklığı (IQR): Üst ve alt çeyrekler arasındaki farktır. Veri setindeki orta %50'lik kısmın yayılımını gösterir

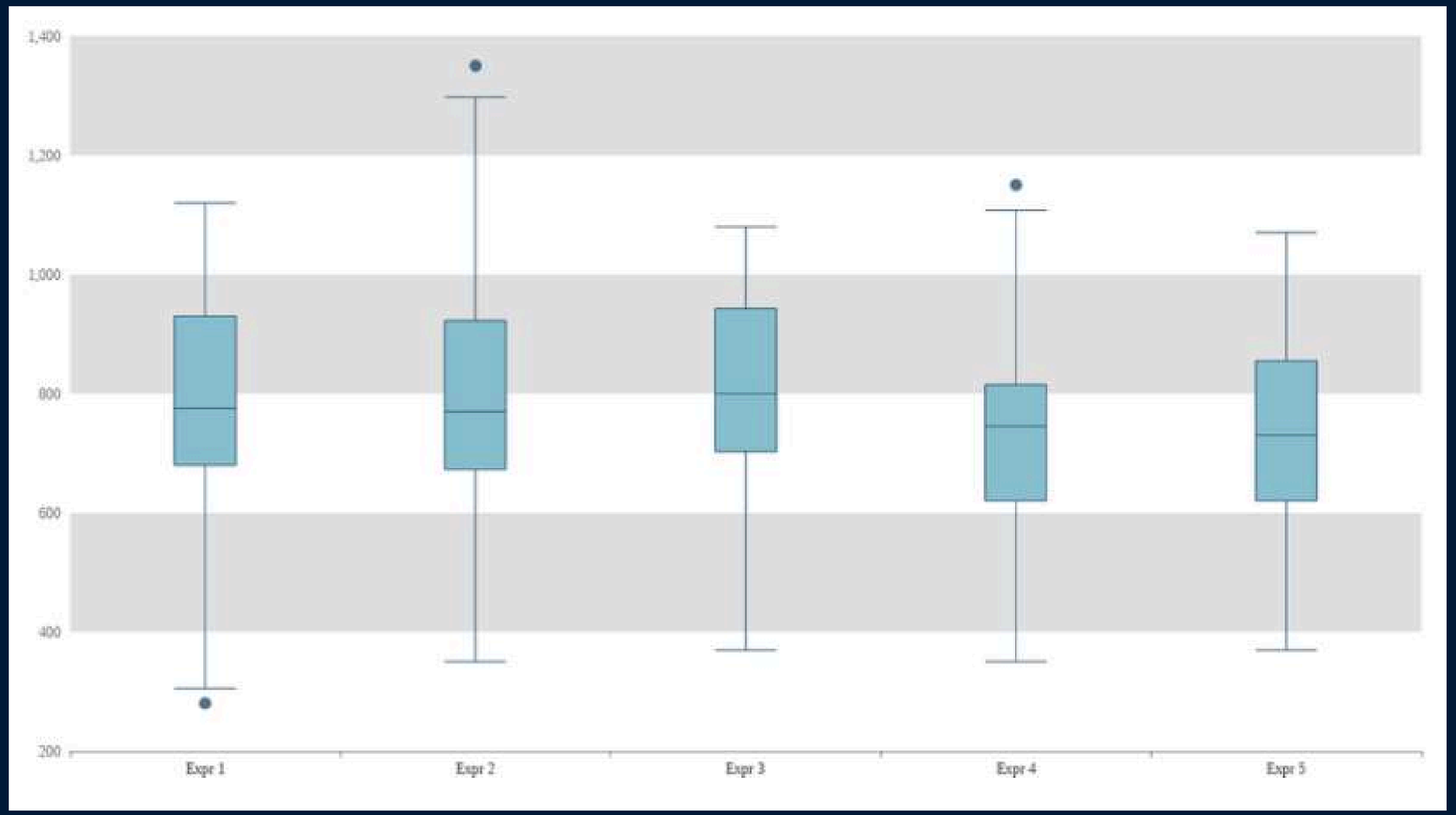
Histogram Grafikleri

Boy Uzunluğu	Sayı	Yüzde(%)
90-92	3	4.0
93-95	5	6.7
96-98	8	10.7
99-101	12	16.0
102-104	14	18.7
105-107	11	14.7
108-110	9	12.0
111-113	8	10.7
114-116	5	6.7
Toplam	75	100.0



Histogram grafiği ile verinin frekans dağılımı,çarpıklık gibi özelliklerini görebiliriz

Kutu Grafikleri (Box Plots)



Kutu grafiği ile verinin medyan, çeyrek ve aykırı değerlerini görsel olarak inceleyebiliriz

Veri Kalitesi Nedir?

Veri kalitesi, bir verinin belirli bir amaca uygunluk derecesini ifade eder. Yani veri ne kadar doğru, eksiksiz, güncel ve tutarlıysa kalitesi o kadar yüksektir. Zayıf veri kümesi verilerin analizi aşamasında gürültülü , eksik , tekrarlı , yanlış veriler gibi sorunlar doğurabilir.

Veri Kalitesi Etkileyen Başlıca Faktörler

Doğruluk

Veri gerçeği ne kadar yansıtıyor?

Tutarlılık

Veriler birbiriyle uyumlu mu?

Güncellik

Veri ne kadar yeni?

Eksiksizlik

Tüm gerekli bilgiler mevcut mu?

Anlaşılırlık

Veri açık ve yorumlanabilir mi?

Eksik Veri

Eksik veri, veri setinde bulunması gereken bilginin mevcut olmamasıdır.Eksik veriler doğru şekilde ele alınmazsa, analiz sonuçlarını yanıltabilir, önyargılı yorumlara yol açabilir ve makine öğrenmesi modellerinin doğruluğunu düşürebilir. Bu nedenle eksik veri analizi ve uygun doldurma yöntemleri, veri ön işleme sürecinin kritik adımlarından biridir.

Eksik veri türleri:

- MCAR: Tamamen rastgele eksik
- MAR: Başka değişkenlere bağlı eksik
- MNAR: Değerin kendisine bağlı eksik

Eksik Verileri Doldurma Yöntemleri

Silme Yöntemleri

Kayıt (Satır) Silme: Eksik veri içeren satırlar tamamen çıkarılır.
Değişken (Sütun) Silme: Eksik veri oranı çok yüksek olan sütunlar tamamen kaldırılır.
Dezavantaj: Çok veri kaybına yol açabilir, örneklem küçülür

Zaman Serilerinde Eksik Veri Doldurma

Önceki Değerle Doldurma (Forward Fill): Eksik noktaya en yakın önceki değer atanır.
Sonraki Değerle Doldurma (Backward Fill): Eksik noktaya en yakın sonraki değer atanır.
Interpolasyon: Eksik noktalar, çevresindeki değerler arasında matematiksel tahminle doldurulur.

Basit Doldurma Yöntemleri

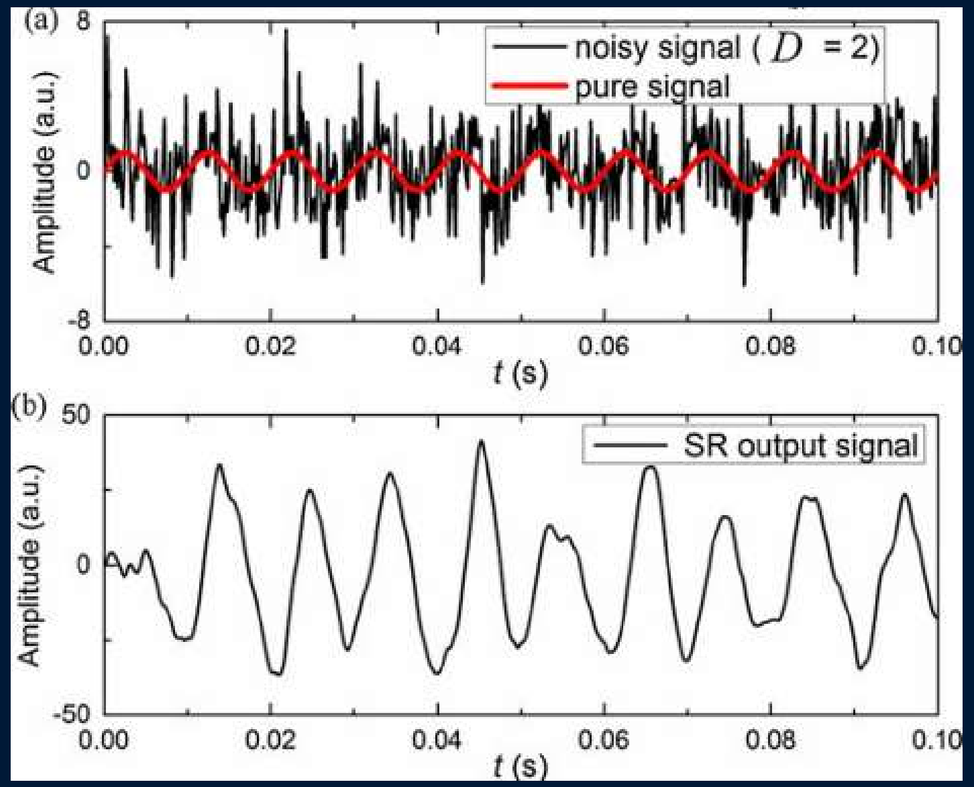
Sabit Değer ile Doldurma: Eksik hücrelere belirlenen sabit bir değer (ör. 0, “Bilinmiyor”) yazılır. Ortalamayla Doldurma: Sayısal değişkenlerde, eksik hücreler o sütunun ortalaması ile doldurulur. Ortanca (Median) ile Doldurma: Özellikle uç değerlerin çok olduğu durumlarda tercih edilir.
Mod ile Doldurma: Kategorik değişkenlerde en sık görülen değer kullanılır

Gelişmiş Doldurma Yöntemleri

K-En Yakın Komşu (KNN) İmputasyonu: Eksik değerler, benzer diğer gözlemlerden tahmin edilir.
Regresyon ile Doldurma: Eksik değer, diğer değişkenler kullanılarak tahmin edilir.
Çoklu İmputasyon (Multiple Imputation): Birden fazla tahmin modeli kullanılarak birden fazla tamamlanmış veri seti üretilir, ardından sonuçlar birleştirilir.

Gürültülü Veri

Gürültülü veri, veri setindeki gerçek bilginin üzerine binen rastgele, düzensiz ve genellikle anlamsız sapmaları ifade eder. Bu tür sapmalar çoğunlukla ölçüm hatalarından, veri girişindeki tutarsızlıklardan ya da çevresel faktörlerin etkilerinden kaynaklanır.



Gürültü, verilerin analiz edilmesini zorlaştırabilir çünkü gerçek desenleri veya eğilimleri gizleyebilir.

Bu nedenle, gürültülü veriler genellikle ön işleme (preprocessing) adımında tespit edilir ve filtreleme, düzeltme ya da çıkarma yöntemleriyle analizden önce temizlenir.

Gürültülü Veri Örnekleri

Sensör Verilerinde Gürültü

Rüzgar, titreşim veya elektromanyetik girişim nedeniyle sensör okumalarının dalgalanması

Ses Tanıma Sistemlerinde Gürültü

Konuşma sırasında arka planda araba kornası, insan sesi ya da müzik gibi seslerin mikrofona gelmesi

Görüntü İşlemede Gürültü

Kamera ile çekilen bir fotoğrafta, ışığın yetersiz olması sonucu oluşan “karlı” görüntü

Anket ve Form Verilerinde Gürültü

Katılımcının rastgele, tutarsız cevaplar vermesi veya zorunlu alanların "asdf" gibi geçersiz veriyle doldurulması

Finansal Verilerde Gürültü

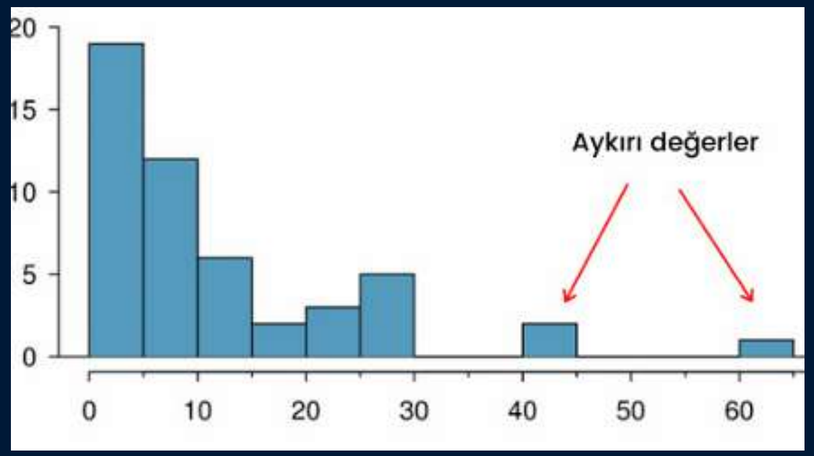
Hisse senedi fiyatlarında bir gün içinde ani, anlamsız sıçramalar (çok küçük zaman aralıklarındaki dalgalanmalar)

Zaman Serilerinde Gürültü

GPS verilerinde birkaç saniyelik tutarsız koordinatlar

Aykırı Veri

Aykırı veri, bir veri kümesinde yer alan ve diğer gözlemlerden belirgin şekilde farklı olan verilerdir. Bu veriler genellikle veri kümesindeki dağılımın dışında yer alır ve istatistiksel analizlerde ortalama, varyans gibi ölçütleri bozabilir. Aykırı değerler; veri giriş hatası, ölçüm sorunu veya gerçekten istisnai bir durumun göstergesi olabilir.



->Bir sınıfta notlar genelde 60-90 aralığında iken, bir öğrencinin 5 alması veya 100 alması 10 yaşındaki çocukların boyu ortalama 140 cm iken birinin boyu 200 cm çıkması
->Bir araç 50 km/s hızla giderken, bir anda 300 km/s gösteren konum verisi
gibi veriler aykırı veriye örnek verilebilir

Aykırı değerleri belirlemenin yöntemleri 3 kategoriye ayrılır; İstatiksel Yöntemler, Görsel Yöntemler, Algoritmik Yöntemler.

İstatistiksel Yöntemler

Z-Skoru (Standart Sapma Yöntemi)

Verinin ortalamadan kaç standart sapma uzakta olduğunu ölçer.

$$z = \frac{x-\mu}{\sigma}$$

X → veri değeri
μ → Ortalama
σ → Standart sapma
|Z|>3 olduğunda o veri aykırı veri kabul edilir

IQR (Çeyrekler Arası Aralık)Yöntemi

Veriyi sıralayıp, ortadaki %50'lik kısmın yayılımına göre uç değerleri bulmak

$$IQR = Q3 - Q1$$

Q1 → Verinin %25'lik kısmının alt değeri (1. çeyrek)
Q3 → Verinin %75'lik kısmının üst değeri (3. çeyrek)
Alt sınır = Q1 – 1.5 × IQR
Üst sınır = Q3 + 1.5 × IQR
bu sınırların dışında kalan değerler aykırı değer olarak kabul edilir

Görsel Yöntemler

Boxplot (Kutu Grafiği)

Verinin medyanını, çeyreklerini (Q1 ve Q3) ve aykırı değerleri tek bir grafikte gösterir.

Ortadaki kutu → Q1 ile Q3 arasındaki değerler Kutunun içindeki çizgi → medyan "Whisker" (çizgi kolları) → uç değer sınırları

Özellikle veri setinde uç noktaların nerede olduğunu görmek için kullanılır.

Hızlı ve anlaşılır, tek değişken için ideal bir yöntemdir

Scatter Plot (Saçılım Grafiği)

İki değişken arasındaki ilişkiyi noktalarla gösterir. Aykırı değerler genelde bu noktaların genel kümesinden uzak olarak görünür.

İlişkiyi ve aykırı değerleri aynı anda görme imkânı veren bir yöntemdir.

Özellikle regresyon analizi veya iki boyutlu verilerde kullanılır

Algoritmik Yöntemler

Isolation Forest

Bu yöntem, veriyi rastgele bölerek aykırı değerleri “izole” etmeye çalışır.

Aykırı değerler, diğerlerinden daha kolay ayrılabildiği için daha kısa bölünme yollarına sahiptir.

Hem sayısal hem kategorik verilerde kullanılabilir.

Sızıntı tespiti, dolandırıcılık tespiti, anomali analizinde kullanılır.

DBSCAN

Bir kümeleme algoritmasıdır. Yoğun bölgelerdeki noktaları küme olarak gruplar, düşük yoğunluklu bölgelerde kalan noktaları “gürültü” yani aykırı değer olarak işaretler

Konum verisi, sensör verisi, uzamsal veri analizlerinde kullanılır

LOF

Her veri noktasının komşularına olan uzaklık yoğunluğunu hesaplar.

Bir nokta, komşularına göre çok daha seyrek bölgede yer alıyorsa aykırı değer kabul edilir.

Özellikle çok boyutlu anomali tespitlerinde kullanılır.

Tekrarlı veriler

Tekrarlı veriler, aynı veri kümesinde birden fazla kez yer alan aynı ya da neredeyse aynı kayıtlardır.
Genellikle veri girişi hataları, sistem birleştirme süreçleri veya eksik kimlik doğrulama nedeniyle oluşur.
Depolama israfı Analizlerde çarpıtılmış sonuçlar gibi sorunlara yol açabilir.

Veri Birleştirme

Veri birleştirme, birden fazla kaynaktan gelen verilerin, tek ve tutarlı bir veri kümesinde birleştirilmesi işlemidir.
Farklı veri formatları, sistemler veya dosyalar arasındaki veriler uyumlu hale getirilir.
Veri analitiği, raporlama ve makine öğrenmesi için sağlam temel sağlar.

Uygulama

İris Veri Seti

```python
import pandas as pd

# Ana veri seti
df1 = pd.DataFrame({
    'id': [1, 2, 3],
    'ad': ['Ahmet', 'Ayşe', 'Mehmet']
})

# Ek veri seti
df2 = pd.DataFrame({
    'id': [1, 2, 3],
    'şehir': ['İstanbul', 'Ankara', 'İzmir']
})

# Birleştirme (id üzerinden)
birlesik_df = pd.merge(df1, df2, on="id", how="left")

print("📌 Birleştirilmiş Veri:")
print(birlesik_df)
```

```
📌 Birleştirilmiş Veri:
   id      ad     şehir
0   1   Ahmet  İstanbul
1   2    Ayşe    Ankara
2   3  Mehmet     İzmir
```

Uygulama

İris Veri Seti

```python
import pandas as pd
import numpy as np

# Örnek veri seti
df = pd.DataFrame({
    'ad': ['Ahmet', 'Ayşe', 'Mehmet'],
    'şehir': ['İstanbul', np.nan, 'İzmir']
})

print("📌 Boş Değerler Öncesi:")
print(df)

# Boş değerleri "Bilinmiyor" ile doldurma
df['şehir'] = df['şehir'].fillna('Bilinmiyor')

print("\n📌 Boş Değerler Doldurulduktan Sonra:")
print(df)
```

```
📌 Boş Değerler Öncesi:
      ad     şehir
0  Ahmet  İstanbul
1   Ayşe       NaN
2 Mehmet     İzmir

📌 Boş Değerler Doldurulduktan Sonra:
      ad       şehir
0  Ahmet    İstanbul
1   Ayşe  Bilinmiyor
2 Mehmet       İzmir
```

Uygulama

İris Veri Seti

```python
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris

# 1 Iris veri setini yükle
iris = load_iris()
df1 = pd.DataFrame(iris.data, columns=iris.feature_names)

# 2 Etiket sütununu ekle
df1['target'] = iris.target

# 3 İkinci bir veri seti oluştur (bazı eksik veriler var)
df2 = pd.DataFrame({
    'region': ['Europe', 'Asia', None, 'America', 'Africa'] * 30,
    'year': [2020, 2021, 2022, None, 2023] * 30
})

# 5 Veri birleştirme (yanyana)
df = pd.concat([df1, df2], axis=1)


# 6 Eksik verileri doldurma
df['region'] = df['region'].fillna('Unknown')
df['year'] = df['year'].fillna(df['year'].mean())

# 7 Son hali görüntüle
print("\nİlk 5 satır:")
print(df.head(5))
```

```
İlk 5 satır:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  target   region    year
0                5.1               3.5                1.4               0.2       0   Europe  2020.0
1                4.9               3.0                1.4               0.2       0     Asia  2021.0
2                4.7               3.2                1.3               0.2       0  Unknown  2022.0
3                4.6               3.1                1.5               0.2       0  America  2021.5
4                5.0               3.6                1.4               0.2       0   Africa  2023.0
```

Kesikli Hale Getirme (Discretization)

Sürekli verilerin kesikli hale dönüştürülmesi işlemidir.
Bu yöntem, verinin anlamlı kategorilere bölünmesini sağlar ve bazen model başarısını artırır.

Eşit genişlik (equal-width), eşit frekans (equal-frequency), k-means temelli discretization gibi yöntemler kullanılır.

Birçok sınıflandırma algoritması kesikli veriler ile daha iyi ve doğru sonuçlar verirler.

İkili Hale Getirme (Binarization)

Sürekli yada kategorik değişkenin ikili (1-0) sisteme dönüştürülmesidir.

Genellikle birliktelik kuralları için gerçekleştirilmektedir.Sürekli veriler önce kategorik verilere daha sonra ikili değerlere çevrilir.

Bu yöntem özellikle lojistik regresyon, karar ağaçları ve sinir ağlarında kullanılır.

Ayrıca kategorik veriler one-hot encoding ile ikili vektörlere dönüştürülür.

Veri Ön İşlemede Ölçeklendirme Teknikleri

Normalizasyon (Min-Max Scaling)

- Verileri belirli bir minimum ve maksimum değere (genellikle 0 ile 1 arasında) dönüştürür.
- Her sütunun en yüksek değerinden en düşük değeri çıkarın ve aralığa bölünerek bulunur. Her yeni sütunun en düşük değeri 0, en yüksek değeri ise 1'dir.
- Sezonsal etki gibi istenmeyen özellikler ortadan kalkar.
- Özellikle veriler farklı ölçeklerdeyse, onları aynı aralığa getirerek modellerin performansını artırırmakta önemli rol oynar

$$z = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Normalizasyon (Min-Max Scaling)

- Verileri ortalaması 0, standart sapması 1 olacak şekilde dönüştürür.
- Aykırı değerlere karşı normalizasyondan daha dayanıklıdır.

$$z = \frac{x_i - \mu}{\sigma}$$

- μ -> ortalama
  σ → standart sapma

PCA

PCA, yani Başlıca Bileşen Analizi, yüksek boyutlu verileri daha az sayıda bileşene indirgemek için kullanılan bir yöntemdir. Bu sayede veri setindeki önemli bilgiyi koruyarak, verinin boyutunu azaltır ve görselleştirmeyi kolaylaştırır. Ayrıca, gürültüyü azaltıp modellerin performansını artırmaya yardımcı olur. Hem hesaplama maliyetini azaltır hem de veriyi görselleştirmeyi kolaylaştırır

Nasıl Kullanılır?

- Verinin varyansını maksimize eden doğrultuları (bileşenleri) bulur.
- İlk bileşen, verideki en yüksek varyansı taşır, ikinci bileşen ise birinciden bağımsız en yüksek ikinci varyansı taşır ve böyle devam eder.
- Bu bileşenler doğrusal birleşimlerdir ve orijinal değişkenlerden türetilir.
- Boyut indirgeme, Gürültü azaltma, Veri görselleştirme, Ön işleme olarak modelin performansını artırmada kullanılır.

PCA

PCA algoritmasının avantajları nelerdir?

- Korele olan sütunları yok eder
- Algoritma performansını artırır. (Çok boyutlu data ile model yavaş çalışabilir. PCA ile model performansı artar)
- Overfit olmayı azaltır.
- Görselleştirme imkanı kazandırır.

PCA algoritmasının dezavantajları nelerdir?

- Data normalizasyonu kesinlikle PCA'den önce yapılmalıdır.
- Boyut azaltılmasından dolayı, orjinal dataya kıyasla bir miktar bilgi kaybı olmaktadır.
- Bağımsız değişkenler daha az yorumlanabilir hale gelir. Çünkü PCA'yı veri kümesine uyguladıktan sonra, orijinal özellikleriniz temel bileşenlere dönüşecektir. Temel Bileşenler, orijinal özelliklerimizin doğrusal birleşimidir (lineer combinasyonudur). Temel Bileşenler, orijinal özellikler kadar okunabilir ve yorumlanabilir değildir.

PCA

PCA bir feature selection tekniği midir?

PCA, featurelardan, lineer kombinasyonlarını tespit ederek, farklı featurelar oluşturur ve datayı farklı bir şekilde temsil etmemizi sağlar. Yani bir feature selection tekniği değildir.

PCA analizi tekniği nedir?

Ölçeklendirme, PCA'nın hesaplanma şekli nedeniyle önemlidir. Verileri kare anlamında en iyi temsil eden lineer alt uzayları bulan Tekil Değer Ayrıştırma yoluyla hesaplanır. Ölçekleme önemlidir, çünkü tekil değer ayrıştırması kareler anlamında yaklaşıktır. Dolayısıyla, bir değişken diğerinden farklı bir ölçekte ise, PCA prosedürüne hakim olacaktır.

Büyük datasetlerde PCA algoritması kullanılır mı?

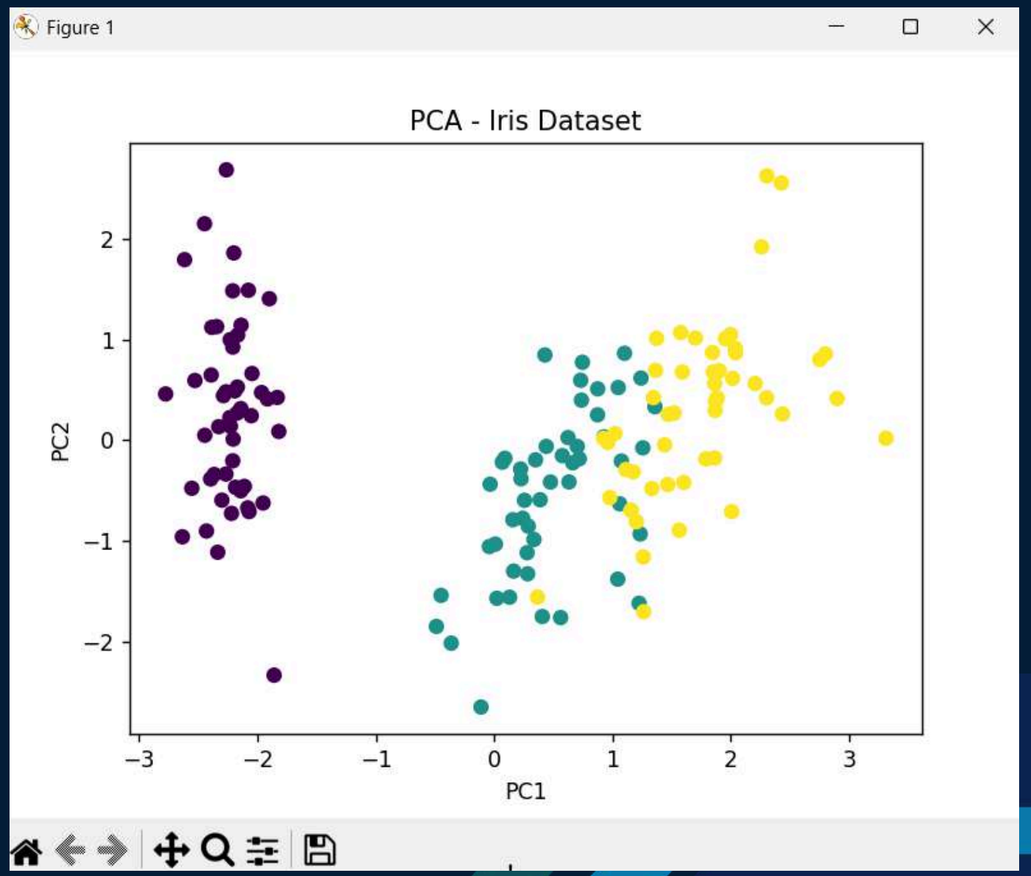
PCA nesnesi çok kullanışlıdır ancak büyük veri kümeleri için belirli sınırlamaları vardır. En büyük sınırlama, PCA'de işlenen tüm verilerin ana belleğe sığması gerekliliğidir. Büyük veri kümesiyle kullanmak için daha iyi bir alternatif IncrementalPCA'dır, bu nesne farklı bir işleme biçimi kullanır ve verileri mini batch biçiminde işlerken PCA'nın sonuçlarıyla neredeyse tam olarak eşleşen kısmi hesaplamalara izin verir.

UYGULAMA:PCA(Iris Veri Seti)

uygulama3.py

```python
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import pandas as pd

iris = load_iris()
X = iris.data
y = iris.target

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print("Özdeğerler:", pca.explained_variance_)
print("Açıklanan Varyans Oranı:", pca.explained_variance_ratio_)
print("Toplam Açıklanan Varyans:", sum(pca.explained_variance_ratio_))

import matplotlib.pyplot as plt

plt.figure()
plt.scatter(X_pca[:,0], X_pca[:,1], c=y)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA - Iris Dataset")
plt.show()
```

```
Özdeğerler: [2.93808505 0.9201649 ]
Açıklanan Varyans Oranı: [0.72962445 0.22850762]
Toplam Açıklanan Varyans: 0.9581320720000166
```



UYGULAMA:Normalizasyon

```python
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.decomposition import PCA
import numpy as np

# Örnek veri (5 örnek, 3 özellik)
X = np.array([
    [10, 200, 0.5],
    [15, 180, 0.7],
    [7, 210, 0.2],
    [20, 190, 0.9],
    [13, 195, 0.4]
])

# MinMaxScaler ile normalizasyon
min_max_scaler = MinMaxScaler()
X_minmax = min_max_scaler.fit_transform(X)
print("MinMaxScaler ile normalizasyon:\n", X_minmax)
```

```
MinMaxScaler ile normalizasyon:
 [[0.23076923 0.66666667 0.42857143]
 [0.61538462 0.         0.71428571]
 [0.         1.         0.        ]
 [1.         0.33333333 1.        ]
 [0.46153846 0.5        0.28571429]]
```

UYGULAMA:Standardizasyon

```python
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.decomposition import PCA
import numpy as np

# Örnek veri (5 örnek, 3 özellik)
X = np.array([
    [10, 200, 0.5],
    [15, 180, 0.7],
    [7, 210, 0.2],
    [20, 190, 0.9],
    [13, 195, 0.4]
])
# StandardScaler ile standardizasyon
standard_scaler = StandardScaler()
X_standard = standard_scaler.fit_transform(X)
print("\nStandardScaler ile standardizasyon:\n", X_standard)
```

```
StandardScaler ile standardizasyon:
 [[-0.67763093  0.5        -0.16552118]
 [ 0.45175395 -1.5         0.66208471]
 [-1.35526185  1.5        -1.40693001]
 [ 1.58113883 -0.5         1.4896906 ]
 [ 0.          0.         -0.57932412]]
```

UYGULAMA:Standardizasyon

```python
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.decomposition import PCA
import numpy as np

# Örnek veri (5 örnek, 3 özellik)
X = np.array([
    [10, 200, 0.5],
    [15, 180, 0.7],
    [7, 210, 0.2],
    [20, 190, 0.9],
    [13, 195, 0.4]
])

standard_scaler = StandardScaler()
X_standard = standard_scaler.fit_transform(X)

# PCA ile boyut indirgeme (2 bileşene indirgeme)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_standard)
print("\nPCA ile boyut indirgeme sonucu:\n", X_pca)
```

```
PCA ile boyut indirgeme sonucu:
 [[-0.7721068   0.06922206]
 [ 1.48482142 -0.8229622 ]
 [-2.45665813  0.17488424]
 [ 2.08673263  0.78346902]
 [-0.34278912 -0.20461312]]
```